

Estado

MOV786205 – CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br



INSTITUTO FEDERAL
Santa Catarina
Campus São José

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Assuntos da aula

<https://drive.google.com/drive/folders/1DoNy5fsXJBMNZGRydM0TVKoAD83CILTU>

- Como interagir com botões usando *Jetpack Compose*
- Como manter o estado de um aplicativo

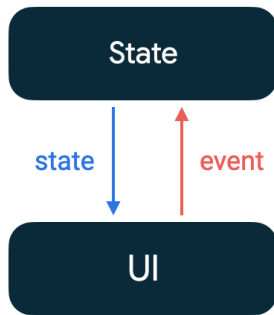
```
Column(  
    modifier = modifier,  
    horizontalAlignment = Alignment.  
        CenterHorizontally  
) {  
    Image( /* detalhes */)  
    Button(onClick = { /*TODO*/ }) {  
        Text("Jogar")  
    }  
}
```



Fluxo de dados unidirecional

Unidirectional Data Flow (UDF)

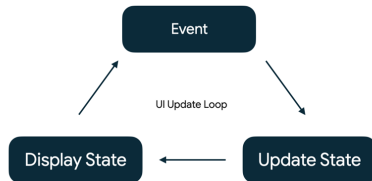
- Dados fluem em uma única direção, do estado para a interface do usuário
- Permite desacoplar a lógica de negócios da interface do usuário



Fonte: (Google, 2025)

Laço de atualização da interface do usuário

- **Evento:** gerado na interface
 - Um clique de botão, informações vindo da rede
- **Atualizar estado:** Um manipulador de eventos pode alterar o estado
- **Exibir estado:** O detentor do estado repassa o estado para baixo, e a interface o exibe

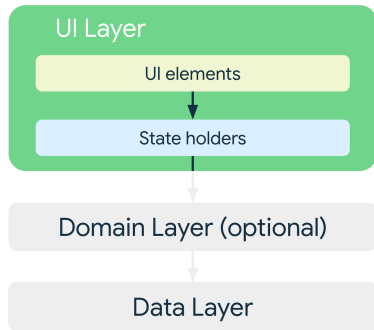


Fonte: (Google, 2025)

Camada de interface

A *UI* é uma representação visual do estado do app recuperado da camada de dados

- Dados do app recebidos da camada de dados costumam estar em um formato diferente das informações que precisam ser mostradas
- A camada de UI é o *pipeline* que converte as mudanças de dados do app em um formato que a UI pode usar para que elas sejam mostradas



Fonte: (Google, 2025)

Estado da UI gerenciado pela própria UI

- Ao clicar no botão, o valor do contador é alterado
- Porém, a tela não reflete essa mudança, pois o **contador não está sendo observado**

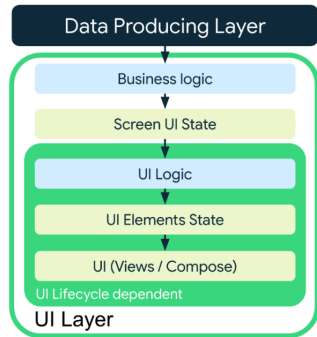
```
@Composable
fun Hidratacao(modifier: Modifier = Modifier) {
    Column(modifier = modifier.padding(16.dp)) {
        var contador = 0 // Estado não observável

        Text("Você bebeu $contador copos de água hoje!")
        Button(
            onClick = {
                contador++
                Log.d("Hidratacao", "Você bebeu $contador copos")
            }, Modifier.padding(top = 8.dp),
            enabled = contador < 10
        ) {
            Text("Beber água")
        }
    }
}
```

Estado da UI gerenciado pela própria UI (*UI logic*)

<https://developer.android.com/develop/ui/compose/state>

- `State<T>` e `MutableState<T>` tornam o estado observável no Compose
- Mudanças em `MutableState` causam recomposição automática dos composables dependentes
- `remember` mantém valores durante recomposições; `rememberSaveable` também persiste em mudanças de configuração (ex: rotação de tela)
- Objetos criados fora do ciclo de recomposição são ser descartados



Fonte: (Google, 2025)

Estado da UI gerenciado pela própria UI

Execute a aplicação e clique no menu *Tools->Layout Inspector* do Android Studio

```
@Composable
fun Hidratacao(modifier: Modifier = Modifier) {
    Column(modifier = modifier.padding(16.dp)) {
        // Estado da UI gerenciado pela própria UI. contador é observado. Inicia em 0
        var contador by remember { mutableStateOf(0) }

        Text("Você bebeu $contador copos de água hoje!")
        Button(
            onClick = {
                contador++
                Log.d("Hidratacao", "Você bebeu $contador copos")
            }, Modifier.padding(top = 8.dp),
            // Lógica da UI para UI, habilitar/desabilitar o botão
            enabled = contador < 10
        ) {
            Text("Beber água")
        }
    }
}
```

Elevação de estado

Elevação de estado I

- Consiste em um padrão onde o estado é movido para um nível superior na árvore de composição, permitindo que componentes filhos compartilhem o mesmo estado
- Isso é útil para evitar a duplicação de estado e facilitar a comunicação entre componentes
 - Uma única fonte de verdade para o estado
- Uma composição sem elevação de estado pode levar a problemas de sincronização e dificuldade na manutenção do código

Elevação de estado II

- A elevação de estado pode ser feita utilizando parâmetros de função e callbacks para passar o estado e as atualizações entre componentes
 - O valor atual a ser mostrado
 - `value: T`
 - O evento que solicita a mudança do valor para um novo valor `T`
 - `onValueChange: (T) -> Unit`

```

@Composable
fun HidratacaoSemEstado(
    contador: Int,
    onValueChange: () -> Unit, // Callback para beber água
    modifier: Modifier = Modifier
) {
    Column(modifier = modifier.padding(16.dp)) {
        Text("Você bebeu $contador copos hoje!")
        Button(
            onClick = { onValueChange() }, enabled = contador < 10
        ) {
            Text("Beber")
        }
    }
}

@Composable
fun HidratacaoComEstado(modifier: Modifier = Modifier) {
    var contadorAgua by rememberSaveable { mutableIntStateOf(0) }
    var contadorSuco by rememberSaveable { mutableIntStateOf(0) }
    Column(modifier = modifier.fillMaxSize()) {
        HidratacaoSemEstado(contador = contadorAgua, onValueChange = { contadorAgua++ })
        HidratacaoSemEstado(contador = contadorSuco, onValueChange = { contadorSuco++ })
    }
}

```

Regras para elevação de estado

<https://developer.android.com/develop/ui/compose/architecture#udf-compose>

- 1 Eleve o estado até o menor ancestral comum de todos os composables que precisam acessar (ler) esse estado
- 2 Eleve o estado até o nível mais alto onde ele pode ser modificado (escrito) por eventos da interface
- 3 Se múltiplos estados precisam ser atualizados juntos em resposta ao mesmo evento, eleve ambos ao mesmo nível para garantir sincronização

Desafio 02

Desafio 02 - Jogo de dados I

Um jogo de azar utiliza dois dados e segue as regras abaixo

- O jogador lança os dois dados.
- Se a soma das faces dos dados for **7 ou 11**, o jogador **ganha** imediatamente.
- Se a soma for **2, 3 ou 12**, o jogador **perde** imediatamente.
- Se a soma obtida no primeiro lançamento for qualquer outro valor, esse número passa a ser o **"ponto"** do jogador. Nos lançamentos seguintes, o objetivo é obter novamente essa mesma soma para **ganhar**. Caso saia 7 antes de atingir o "ponto", o jogador **perde**.

Desafio 02 - Jogo de dados II

Um jogo de azar utiliza dois dados e segue as regras abaixo

```
Primeiro lançamento:
  2 e 5 = 7
Jogador ganhou!
...
Novo jogo
Primeiro lançamento:
  2 e 6 = 8
Ponto a ser buscado: 8
Segundo lançamento:
  1 e 3 = 4
Terceiro lançamento:
  5 e 2 = 7
Jogador perdeu!
...
Novo jogo
Primeiro lançamento:
  3 e 5 = 8
Ponto a ser buscado: 8
Segundo lançamento:
  4 e 4 = 8
Jogador ganhou!
...
```

Exemplo de sorteio de dado

Carregando imagem de acordo com o valor do dado

```
@Composable
fun Dados(modifier: Modifier = Modifier) {
    // Cria um estado reativo inteiro com valor inicial 1
    // O operador 'by' em Kotlin é utilizado para delegação de propriedades
    // 'face' é uma propriedade delegada a um objeto criado por 'remember { mutableStateOf(1) }'.
    // Isso significa que 'face' irá armazenar um estado mutável que persiste durante recomposições do Composable
    var face by rememberSaveable { mutableStateOf(1) }

    val imagem = when (face) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    // .....
    Image(
        painter = painterResource(imagem),
        contentDescription = face.toString()
    )
    Button(onClick = { face = (1..6).random() }) {
        Text("Jogar", fontSize = 24.sp)
    }
}
```

ViewModel

Backing property

Entendendo a convenção de propriedades em Kotlin

- Em Kotlin, uma *backing property* é uma variável privada que realmente armazena o valor de uma propriedade pública
- Essa abordagem permite controlar como o valor é acessado ou modificado, protegendo o estado interno da classe
- Por convenção, a propriedade privada recebe um nome iniciado por _ (underscore), enquanto a propriedade pública tem o mesmo nome
- Assim, o código externo acessa apenas a propriedade pública, enquanto a lógica interna pode modificar o valor diretamente pela propriedade privada

```
private var _contador = 0
val contador: Int
    get() = _contador
```

Lógica de UI vs. Lógica de Negócios

- **Lógica de UI:** responsável por mostrar as mudanças de estado na tela, como navegação ou exibição de mensagens
- **Lógica de negócios:** define o que acontece quando o estado muda, por exemplo, processar pagamentos ou salvar preferências. Essa lógica fica fora da camada de UI
- **ViewModel:** conecta a interface com a lógica de negócios, fornecendo o estado da UI e acesso às operações necessárias. O ViewModel sobrevive a mudanças de configuração e tem um ciclo de vida mais longo que os composables

Tipos de detentores de estado

■ Detentor de estado da lógica de UI

- Quando o estado é gerenciado diretamente pelos composables, como no exemplo do contador de água

■ Detentor de estado da lógica de negócio

- Quando o estado é gerenciado por uma camada de lógica de negócio, como em um `ViewModel`
- `LiveData` e `StateFlow` são exemplos de detentores de estado observáveis da lógica de negócio

Nota

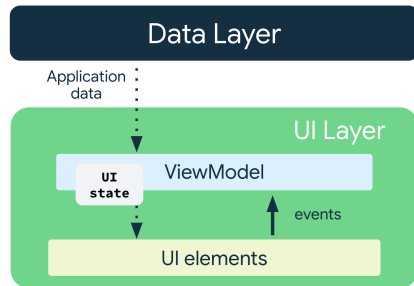
Para a lógica da interface e estados de curta duração, uma classe simples com um ciclo de vida dependente apenas da interface é suficiente. Caso você precise acesso à lógica de negócios e queira que o estado da interface sobreviva enquanto a tela puder ser acessada, considere usar um `ViewModel`.

ViewModel

- Classe que armazena e gerencia os dados da interface, além de encapsular a lógica de negócios do app
- Garante que os dados sobrevivam a mudanças de configuração, como rotação de tela ou navegação entre atividades
- A interface não precisa buscar ou recriar dados toda vez que a tela muda, tornando o app mais eficiente
- Facilita a separação entre a lógica de negócios e a interface do usuário, deixando o código mais organizado e fácil de manter
- Pode ser integrado a outras bibliotecas de arquitetura, como o Navigation, que simplifica a navegação entre diferentes telas do aplicativo

ViewModel e fluxo de dados unidirecional

- 1 A ViewModel retém e expõe o estado a ser consumido pela UI
 - O estado da UI são os dados do app transformados pela ViewModel
- 2 A UI notifica a ViewModel sobre eventos do usuário
- 3 A ViewModel processa as ações do usuário e atualiza o estado
- 4 O estado atualizado é retornado à UI para renderização



Fonte: (Google, 2025)

Como usar o ViewModel no Compose

<https://developer.android.com/develop/ui/compose/architecture#viewmodel-compose>

- Incluir a dependência do ViewModel no arquivo `build.gradle.kts`

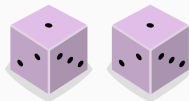
```
dependencies {  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.9.3")  
}
```

Imutabilidade e fonte única de verdade

- O estado da interface deve ser imutável e representado por um único objeto
 - Em Kotlin, usar `data class` facilita criar estados imutáveis
- Objetos imutáveis evitam que diferentes partes do app alterem o estado ao mesmo tempo, reduzindo inconsistências e bugs
- A interface deve apenas ler e mostrar o estado. Ela só pode modificar o estado diretamente quando é a única responsável pelos dados

Exemplo de ViewModel para o jogo de dados

```
data class DadosUiState(  
    val dado1: Int = 1,  
    val dado2: Int = 1,  
    val totalJogadas: Int = 0  
)  
  
class DadosViewModel : ViewModel() {  
    private val _uiState = MutableStateFlow(DadosUiState())  
    val uiState = _uiState.asStateFlow()  
  
    fun jogarDados() {  
        _uiState.update { // Atualiza o estado da UI  
            it.copy( // Fazendo cópia do estado atual, mantendo a  
                imutabilidade  
                dado1 = (1..6).random(),  
                dado2 = (1..6).random(),  
                totalJogadas = it.totalJogadas + 1  
            )  
        }  
    }  
}
```



Jogar Dados

Total de jogadas: 0

Exemplo de ViewModel para o jogo de dados

```
fun imagemDado(dado: Int): Int { /* Retorna id da imagem correspondente ao dado */ }

@Composable
fun Dados(modifier: Modifier = Modifier, viewModel: DadosViewModel = viewModel()) {

    // Coleta o estado atual dos dados do ViewModel
    val uiState by viewModel.uiState.collectAsState()
    Image(
        painter = painterResource(imagemDado(uiState.dado1)),
        contentDescription = "Dado 1"
    )
    Image(
        painter = painterResource(imagemDado(uiState.dado2)),
        contentDescription = "Dado 2"
    )
    Button(onClick = { viewModel.jogarDados() }) { Text("Jogar Dados") }
    Text(
        text = "Total de jogadas: ${uiState.totalJogadas}"
    )
}
```

StateFlow

- StateFlow é uma forma reativa de gerenciar e observar estados em Kotlin
 - Sua propriedade `value` sempre contém o valor mais recente do estado
- A interface pode coletar o StateFlow e atualizar automaticamente quando o estado muda
- Para alterar o estado, basta atribuir um novo valor à propriedade `value` de um `MutableStateFlow`

```
val stateFlow = MutableStateFlow(0)

// Lê o estado atual
val currentState = stateFlow.value

// Atualiza o estado
stateFlow.value = 1
```

Aula baseada em



GOOGLE. **Android Developer**. 2025. Disponível em: <https://developer.android.com/>. Acesso em: 9 jul. 2025.